

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

**Федеральное государственное автономное образовательное учреждение
высшего образования**

«Российский университет дружбы народов имени Патриса Лумумбы»

Факультет физико-математических и естественных наук

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №11

**«Программирование в командном процессоре ОС UNIX.
Ветвления и циклы»**

Студент: Расулов Али НКАбд-06-25

Москва, 2026

1. Цель работы

Изучить основы программирования в оболочке ОС UNIX, научиться писать командные файлы с использованием логических управляющих конструкций (if, case, getopt) и циклов (for, while, until).

2. Описание результатов выполнения задания

Задание 1. Анализ ключей командной строки и поиск строк

Командный файл анализирует ключи командной строки:

- -inputfile – прочитать данные из указанного файла;
- -outputfile – вывести данные в указанный файл;
- -p – указать шаблон для поиска;
- -C – различать большие и малые буквы;
- -n – выдавать номера строк.

Затем выполняется поиск в указанном файле строк, соответствующих шаблону.

Листинг 1: search.sh

```
1  #!/bin/bash
2
3  input=""
4  output=""
5  pattern=""
6  case_sensitive="-i"
7  line_numbers=""
8
9  while getopt "i:o:p:Cn" opt; do
10     case $opt in
11         i) input="$OPTARG" ;;
12         o) output="$OPTARG" ;;
13         p) pattern="$OPTARG" ;;
14         C) case_sensitive="" ;;
15         n) line_numbers="-n" ;;
16         *) echo "Usage: _$0_-i_input_-o_output_-p_pattern_[-C]_[-n]"; exit 1
17             ;;
18     esac
19 done
20
21 if [ -z "$input" ] || [ -z "$pattern" ]; then
22     echo "Error: _input_file_and_pattern_are_required"
```

```

22     exit 1
23 fi
24
25 if [ -n "$output" ]; then
26     grep $case_sensitive $line_numbers "$pattern" "$input" > "$output"
27 else
28     grep $case_sensitive $line_numbers "$pattern" "$input"
29 fi

```

Листинг 1: search.sh

Результат выполнения

Команда:

```
$ ./search.sh -i test.txt -p "hello" -C -n
```

Вывод:

```

3:Hello world
5:hello there

```

Задание 2. Программа на С и анализ кода завершения

Программа на С вводит число и завершается с кодом:

- `exit(1)` – число больше нуля;
- `exit(2)` – число меньше нуля;
- `exit(0)` – число равно нулю.

Командный файл анализирует код завершения через `$?`.

Листинг 2: checknum.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n;
6     printf("Enter number: ");
7     scanf("%d", &n);
8     if (n > 0) exit(1);
9     else if (n < 0) exit(2);
10    else exit(0);

```

```
11 }
```

Листинг 2: checknum.c

Компиляция:

```
$ gcc checknum.c -o checknum
```

Листинг 3: run_check.sh

```
1 #!/bin/bash
2 ./checknum
3 status=$?
4 if [ $status -eq 0 ]; then
5     echo "Number_is_zero"
6 elif [ $status -eq 1 ]; then
7     echo "Number_is_greater_than_zero"
8 elif [ $status -eq 2 ]; then
9     echo "Number_is_less_than_zero"
10 else
11     echo "Unknown_exit_code"
12 fi
```

Листинг 3: run_check.sh

Результат выполнения

```
Enter number: -5
Number is less than zero
```

Задание 3. Создание и удаление нумерованных файлов

Командный файл создаёт файлы 1.tmp, 2.tmp, ..., N.tmp, где N передаётся аргументом. При запуске с ключом -d удаляет все созданные файлы.

Листинг 4: files.sh

```
1 #!/bin/bash
2
3 if [ "$1" = "-d" ]; then
4     rm -f [0-9]*.tmp
5     echo "All_created_files_deleted"
6     exit 0
7 fi
8
9 if [ -z "$1" ]; then
10     echo "Usage: $0 N_or_$0 -d"
```

```

11     exit 1
12 fi
13
14 N=$1
15 for ((i=1; i<=N; i++)); do
16     touch "$i.tmp"
17 done
18 echo "Created_ $N_files: 1.tmp ... $N.tmp"

```

Листинг 4: files.sh

Результат выполнения

```

$ ./files.sh 4
Created 4 files: 1.tmp ... 4.tmp

```

```

$ ls *.tmp
1.tmp 2.tmp 3.tmp 4.tmp

```

```

$ ./files.sh -d
All created files deleted

```

Задание 4. Архивация файлов в директории

Командный файл упаковывает в архив все файлы указанной директории. Модифицированная версия архивирует только файлы, изменённые менее недели назад (используется `find -mtime -7`).

Листинг 5: backup.sh

```

1  #!/bin/bash
2
3  dir=$1
4  arch="backup_$(date +%Y%m%d).tar.gz"
5
6  if [ -z "$dir" ]; then
7      echo "Usage: $0 <directory>"
8      exit 1
9  fi
10
11 #
12 tar -czf "$arch" -C "$dir" . 2>/dev/null
13 echo "Full backup: $arch"
14
15 #
16 find "$dir" -type f -mtime -7 -print0 > /tmp/files.txt
17 tar -czf "recent_$arch" -C "$dir" --null -T /tmp/files.txt 2>/dev/null
18 echo "Recent backup: recent_$arch"

```

Листинг 5: backup.sh

Результат выполнения

```
$ ./backup.sh /home/user/docs
Full backup: backup_20260516.tar.gz
Recent backup: recent_backup_20260516.tar.gz
```

3. Выводы

В ходе выполнения лабораторной работы:

- освоена обработка ключей командной строки с помощью `getopts`;
- закреплено взаимодействие shell-скриптов с программами на C через коды возврата;
- изучены циклы `for`, `while`, `until` и условные операторы;
- применены команды `find` и `tar` для выборочной архивации;
- понята разница между `while` (пока условие истинно) и `until` (пока условие ложно).

Цель работы достигнута полностью.

4. Ответы на контрольные вопросы

- 1. Каково предназначение команды `getopts`?**
`getopts` используется для разбора ключей командной строки в shell-скриптах. Она позволяет обрабатывать опции с аргументами и без, автоматически перебирая их в цикле.
- 2. Какое отношение метасимволы имеют к генерации имён файлов?**
Метасимволы (`*`, `?`, `[]`) служат для *pathname expansion* — замены шаблона списком файлов. Например, `*.txt` раскрывается во все файлы с расширением `.txt`.
- 3. Какие операторы управления действиями вы знаете?**
`if/then/elif/else/fi`, `case/esac`, `for`, `while`, `until`, `break`, `continue`, `exit`, `return`.
- 4. Какие операторы используются для прерывания цикла?**
`break` — полный выход из цикла; `continue` — переход к следующей итерации.
- 5. Для чего нужны команды `false` и `true`?**
`true` возвращает код 0 (успех), `false` — ненулевой код ошибки. Используются для организации бесконечных циклов или задания заведомо истинного/ложного условия.

6. Что означает строка `if test -f man$s/$i.$s`, встреченная в командном файле?

Она проверяет существование файла, имя которого формируется из переменных `$s` и `$i`. Обратный слеш перед `$` экранирует подстановку переменной на текущем уровне. Вероятно, имелось в виду `if test -f man$s/$i.$s`.

7. Объясните различие между конструкциями `while` и `until`.

`while` выполняет тело цикла, **пока** условие истинно.

`until` выполняет тело цикла, **пока** условие ложно (т.е. до тех пор, пока оно не станет истинным).